BICOL UNIVERSITY - POLANGUI
Web Systems and Technology

ALSOR CO.

BS INFORMATION TECHNOLOGY

PROPONENTS:

Ivan Jacob Milan
Nadel Volante
James Espinosa
Erick John Bonaobra
Manuel Angelo Loma

DearBUP
Feel. Write. Share.

Week 14: Phase 5 Displaying and Managing Frontend

INTRODUCTION

The following documentation is the requirements for this week's activity. The screenshots showcased the API response from our MongoDb Atlas which fetches real data from our website.

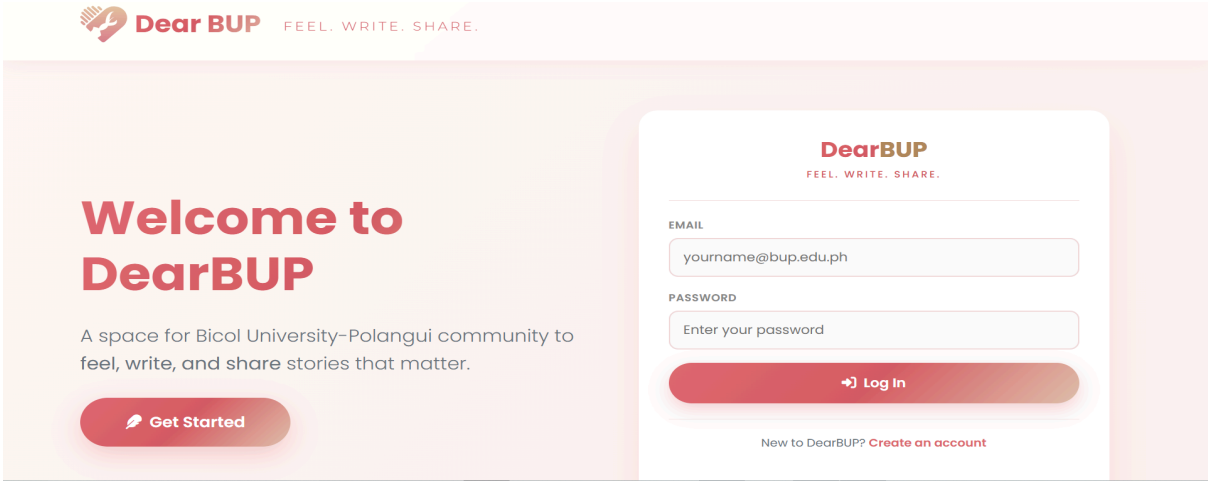
This lab report consists of a fully functional website dashboards and interfaces and responses from every relevant page on our website.

- Log-in Phase
- Home Page
- Organization Page
- Activity Tracker Page
- Profile Page

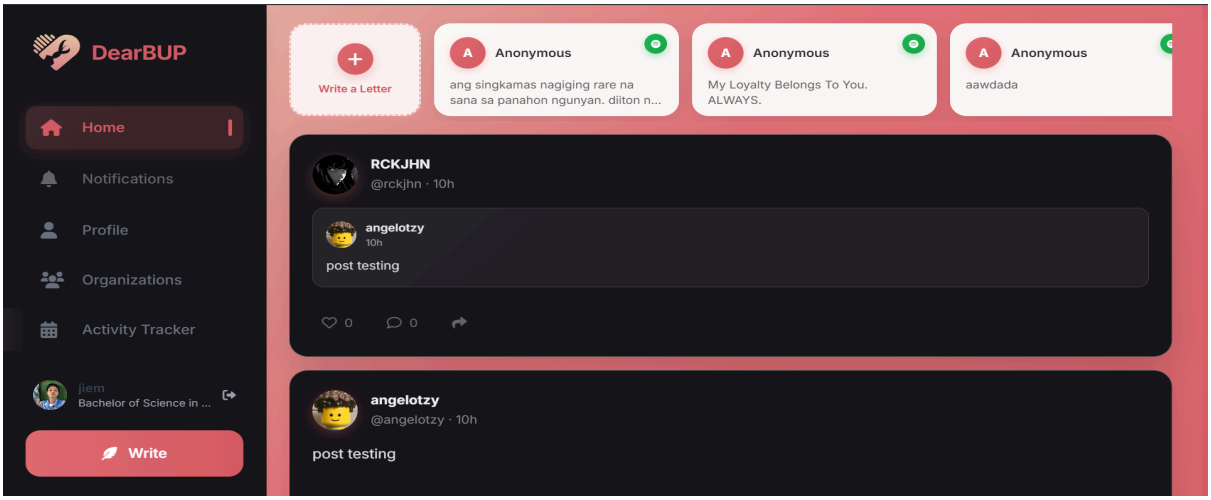
SAMPLE PAGES

The following is the documentation for the working pages available.

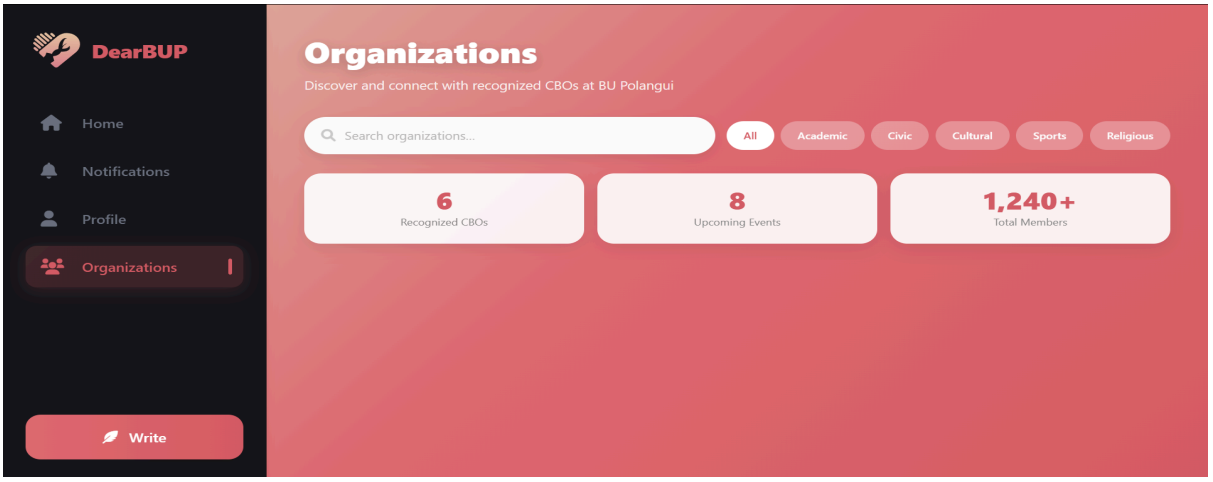
a. Log-in Page



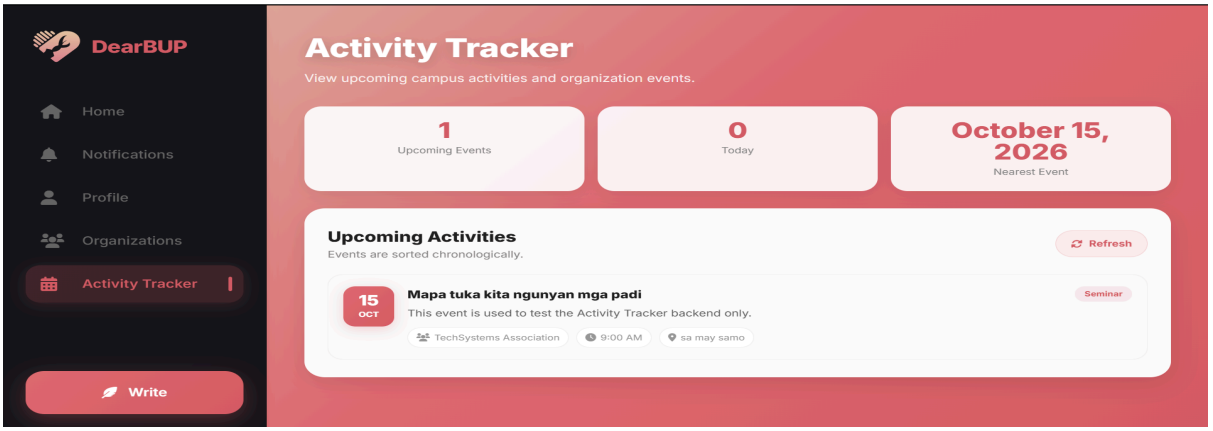
b. Home Page



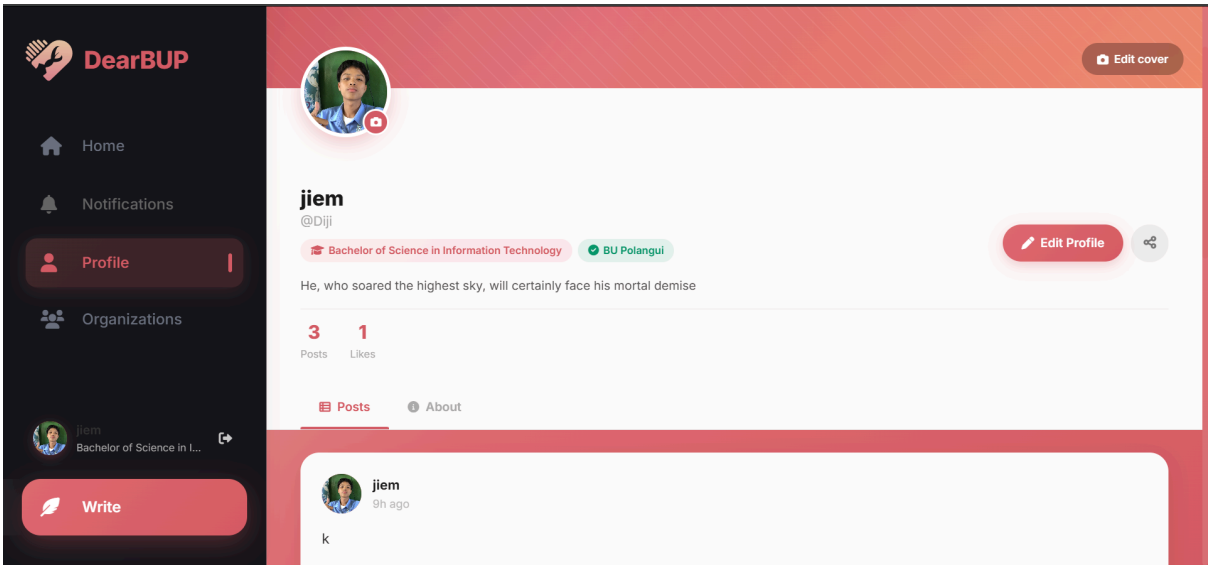
c. Organization Page



d. Activity Tracker Page



e. Profile Page



SAMPLE BACKEND API RESPONSE

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\lenovo\Desktop\WEBSITE\Final Proj\final-project-Alsor-co-BSIT2a> cd backend
PS C:\Users\lenovo\Desktop\WEBSITE\Final Proj\final-project-Alsor-co-BSIT2a\backend> npm run dev

> backend@1.0.0 dev
> nodemon src/server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node src/server.js`
🚀 Server running on http://localhost:5000
🎵 Spotify Login: http://localhost:5000/api/spotify/login
MongoDB Connected: ac-sosa7jq-shard-00-00.dlmsl9e.mongodb.net
```

Our backend connects with the spotify API and it is running on localhost 5000 assigned to the .env. The confirmation message shows that the connection between the database(MongoDB Atlas) and the Spotify API has been successfully integrated.

SAMPLE RESPONSES

1.Register/Log-in response

- Create account

POST http://localhost:5000/api/auth/login — Send

Docs | Params | Authorization | Headers 9 | Body | Scripts | Settings | Cookies

none | form-data | x-www-form-urlencoded | raw | binary | GraphQL | JSON | Schema | Beautify

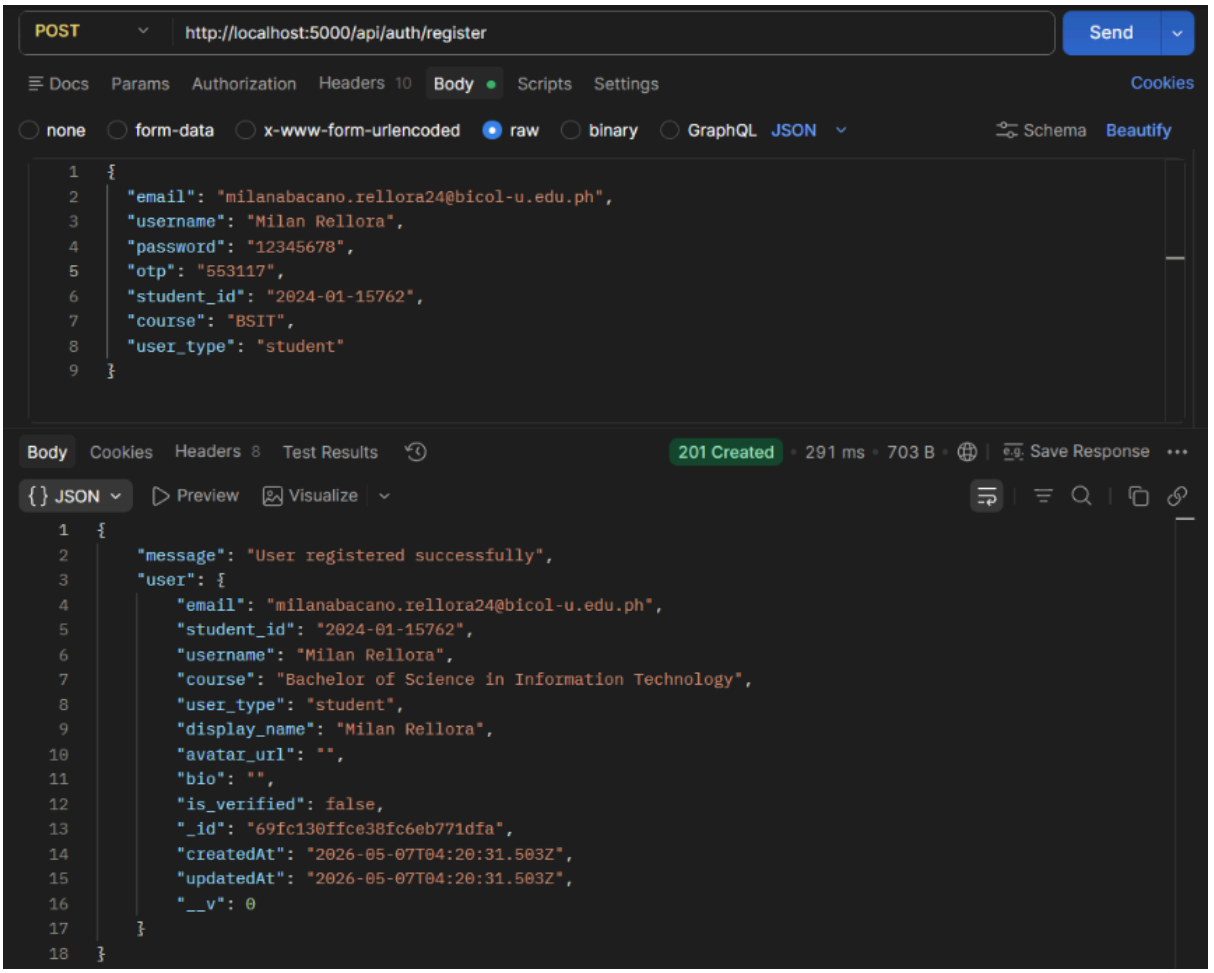
```
{
  "email": "milanabacano.rellora24@bicol-u.edu.ph",
  "password": "12345678"
}
```

Body | Cookies | Headers 8 | Test Results — 200 OK · 133 ms · 890 B — Save Response

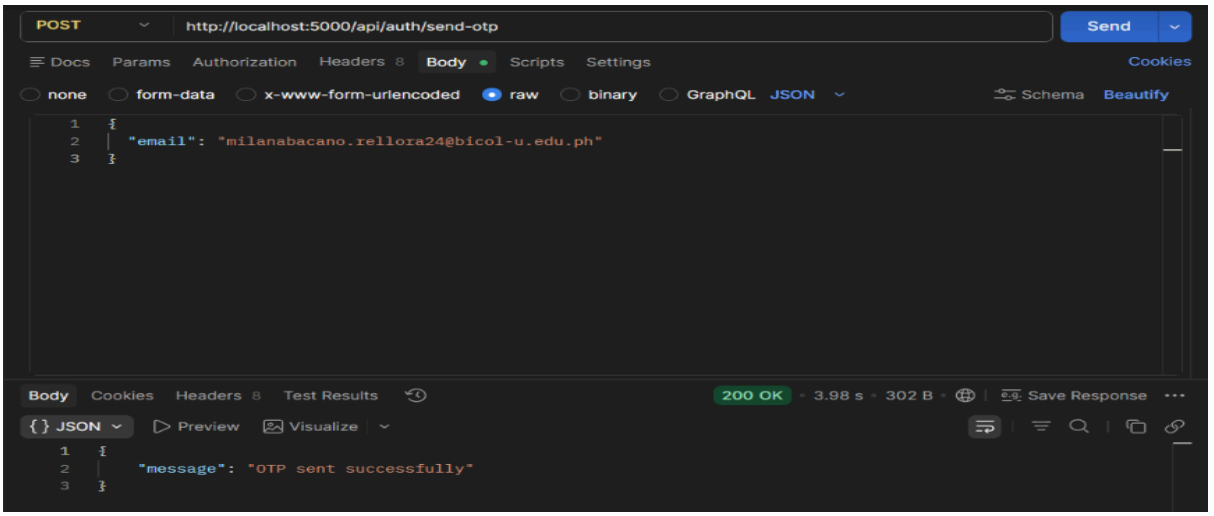
{} JSON | Preview | Visualize


```
{
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.
    eyJpZCI6IjY5ZmMxMzBmZmNlMzhmYzZlYjc3MWRmYSIsInJvbGUiOiJzdHVkZW50IiwiaWF0IjoxNzc4MTI3NjgzLCJleHAiOjE3
    Nzg3MzI0ODN9.u_7Lr8DSjC4vNsrBxhHuxnxhwfzjUWNM_TicuMql55o",
  "user": {
    "_id": "69fc130ffce38fc6eb771dfa",
    "email": "milanabacano.rellora24@bicol-u.edu.ph",
    "student_id": "2024-01-15762",
    "username": "Milan Rellora",
    "course": "Bachelor of Science in Information Technology",
    "user_type": "student",
    "display_name": "Milan Rellora",
    "avatar_url": "",
    "bio": "",
    "is_verified": false,
    "createdAt": "2026-05-07T04:20:31.503Z",
    "updatedAt": "2026-05-07T04:20:31.503Z",
    "__v": 0
  }
}
```

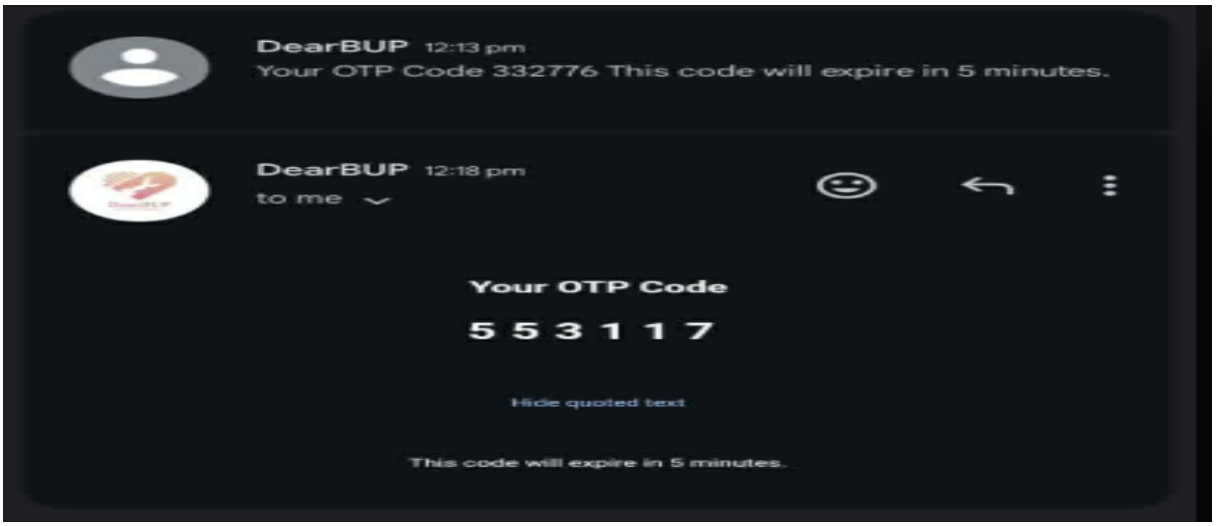
- Register

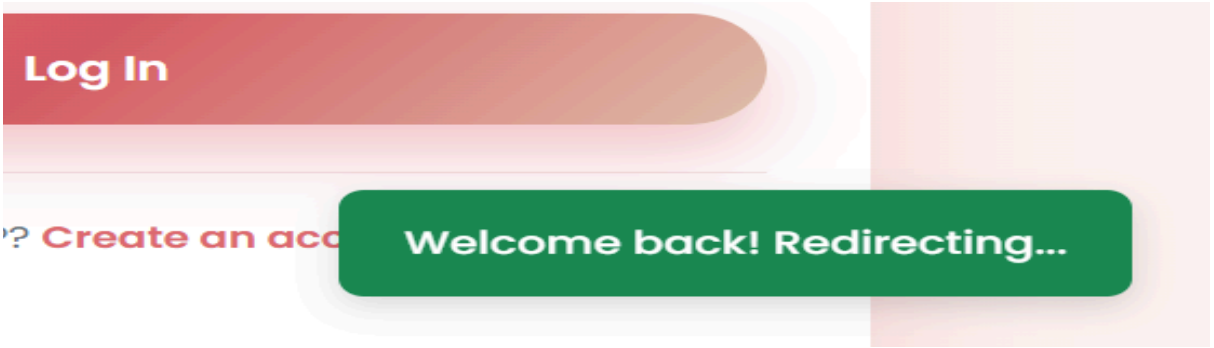


- Confirmation email

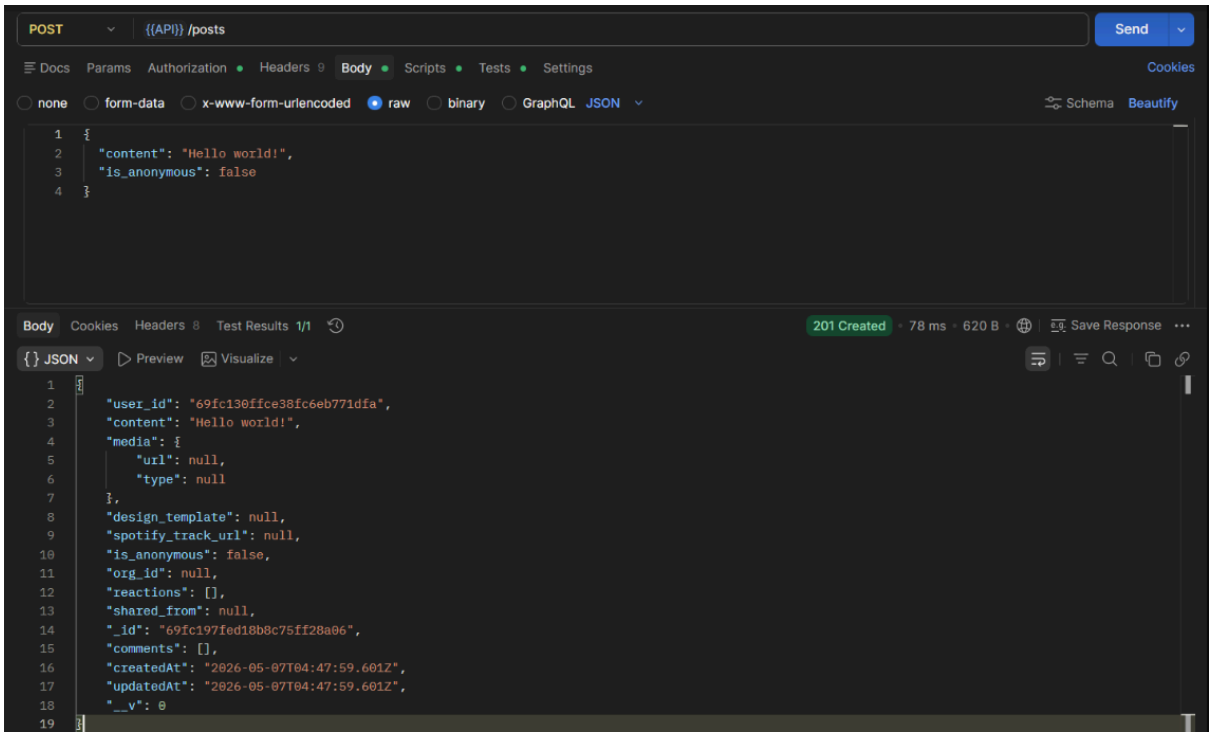


To verify the log-in the user must receive an email containing the OTP or the number digits which can be used to create a new account. Through this confirmation email, our website avoids unverified accounts from the Bicol University community because the web requires an email that ends with a verified bicol-u.edu.ph domain or address.



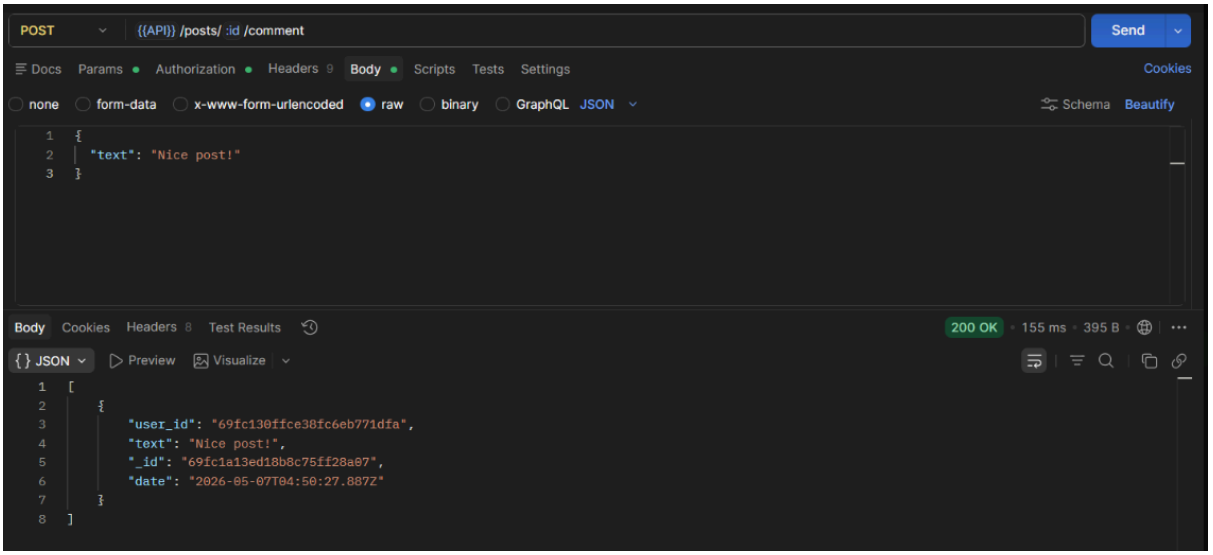


2. Post a letter response
    - To post



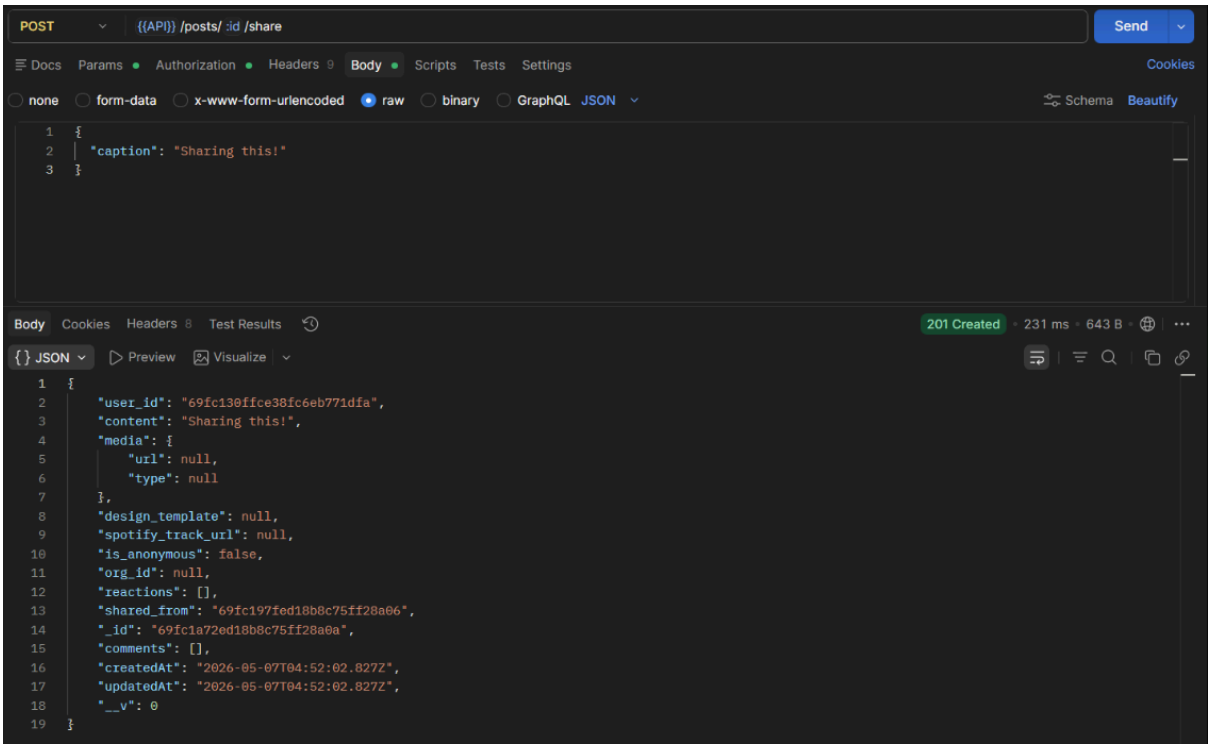
In a single post, there are three things you can do to it. The first one is to post which is directly added to the newsfeed of the user.

- To comment



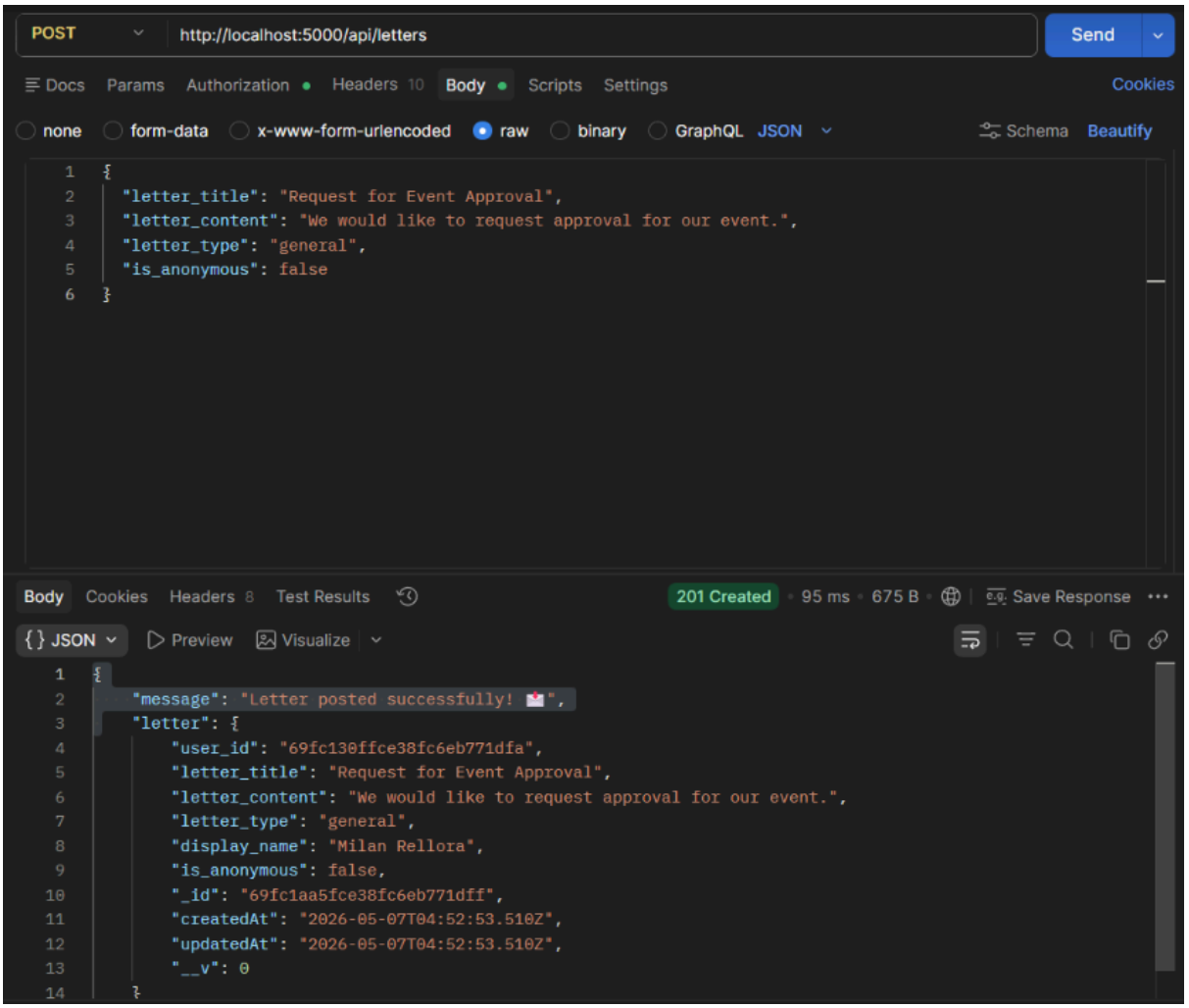
Another user can comment on the post from other users. You can also comment on your own post.

- To share



The user can share posts from other users or they can repost their own letter in their feed.

3. Write a letter response



If you click the feather button that symbolizes “write” your letter automatically sends data to add in the slides of the story like dashboard or the one in the top of the home page with a series of letters.

DATA RENDERING (JS snippet)

1. Load upcoming events

```
 eventList.innerHTML = `
      <div class="tracker-empty">
        Failed to load events. Make sure the backend is running and
/api/events works.
      </div>
    `;

    totalUpcoming.textContent = "0";
    todayEvents.textContent = "0";
    nextEventDate.textContent = "--";
  }
}
function loadSidebarUser() {
  const user = JSON.parse(localStorage.getItem("dearbup_user"));
  if (!user) return;

  const nameEl = document.getElementById("sidebarName");
  const courseEl = document.getElementById("sidebarCourse");
  const avatarEl = document.getElementById("sidebarAvatar");

  if (nameEl) {
    nameEl.textContent = user.display_name || user.username || "User";
  }

  if (courseEl) {
    courseEl.textContent = user.course || "";
  }

  if (avatarEl) {
    if (user.avatar_url) {
      avatarEl.innerHTML = `<img src="${user.avatar_url}"
style="width:100%;height:100%;object-fit:cover;border-radius:50%;" />`;
    } else {
      avatarEl.textContent = (user.display_name || user.username ||
"U")[0].toUpperCase();
    }
  }
}
```

This code snippet manages data rendering by dynamically updating the UI with user information from `localStorage` and displaying error states for event data. It uses `.textContent` and `.innerHTML` to inject values like names and avatars into specific sidebar elements or show a "Failed to load" message if the backend connection fails.

2. Load Letters

```
async function loadLetter() {
    const params   = new URLSearchParams(window.location.search);
    const letterId = params.get('id');
    const main     = document.getElementById('main');

    if (!letterId) {
        main.innerHTML = `
            <div class="state-msg">
                <i class="fas fa-envelope-open"></i>
                <h2>No letter found</h2>
                <p>The letter ID is missing from the URL.</p>
            </div>`;
        return;
    }

    try {
        const res = await fetch(`${API_BASE}/letters/${letterId}`);

        if (res.status === 404) {
            main.innerHTML = `
                <div class="state-msg">
                    <i class="fas fa-trash-alt"></i>
                    <h2>Letter not found</h2>
                    <p>This letter may have been deleted or doesn't
exist.</p>
                </div>`;
            return;
        }

        if (!res.ok) throw new Error();
        const letter = await res.json();
```

This code snippet functions as a **data-fetching and error-handling layer** that determines what the user sees based on the URL and API response. It first checks for a valid `letterId` in the browser's address bar and immediately renders a "No letter found" message if the ID is missing. If an ID exists, it attempts to `fetch` the data from the backend, either displaying a 404 error message if the letter is gone or converting the successful response into a JSON object for further rendering.

3. Render full view of all Letters

```
function renderLetters(letters) {
  const grid = document.getElementById('lettersGrid');

  if (!letters.length) {
    grid.innerHTML = `
      <div class="state-msg">
        <i class="fas fa-envelope-open"></i>
        <p>No letters found in this category.</p>
      </div>`;
    return;
  }

  grid.innerHTML = letters.map((l, i) => {
    const authorName = l.is_anonymous ? 'Anonymous' : (l.display_name
|| l.user_id?.display_name || l.user_id?.username || 'Anonymous');
    const avatarHtml = (!l.is_anonymous && l.user_id?.avatar_url)
      ? `<img src="${escapeHTML(l.user_id.avatar_url)}" alt="" />`
      : escapeHTML(authorName.charAt(0).toUpperCase());

    const type = l.letter_type || 'general';
    const typeLabel = type.charAt(0).toUpperCase() + type.slice(1);
    const hasMusic = !!(l.spotifySongName || l.spotifyTrackUri);

    return `
```

This function serves as the **dynamic UI generator** that transforms an array of data objects into a visual grid of letter cards. It utilizes the `.map()` method to iterate through the letters, conditionally handling logic for anonymous authors, avatars, and category labels. By injecting this processed data into the `lettersGrid` via template literals, it ensures the frontend stays synchronized with the database content.

4. Get notifications

```
/** GET /api/notifications */
async function apiGetNotifications() {
  const res = await fetch(`${API_BASE}/notifications`, {
    headers: authHeaders(),
  });
  if (res.status === 401) {
    localStorage.removeItem("dearbup_token");
    localStorage.removeItem("dearbup_user");
    window.location.href = INDEX_URL;
    return null;
  }
  if (!res.ok) throw new Error(`Fetch failed (${res.status})`);
  return res.json();
}
```

This function handles **authenticated data retrieval** by fetching user-specific alerts from the notification API endpoint. It includes a security check that automatically clears local session data and redirects the user to the login page if an "Unauthorized" (401) status is detected. If the request is successful, it parses the response into a JSON object to be used by the frontend notification system.

6. Search Spotify song

```
renderSpotifyResults(tracks) {
        const listEl            =
document.getElementById('postSpotifyResultsList');
        const resultsContainer =
document.getElementById('postSpotifyResults');
        if (!listEl || !resultsContainer) return;

        listEl.innerHTML = '';
        if (!tracks || tracks.length === 0) {
            resultsContainer.style.display = 'none';
            return;
        }
        resultsContainer.style.display = 'block';

        tracks.forEach(track => {
            const item = document.createElement('div');
            item.className = 'spotify-result-item';

            const albumImg = track.album?.images?.[0]?.url || '';
            const artistName = track.artists?.[0]?.name || 'Unknown
Artist';

            item.innerHTML = `
                ${albumImg
                    ? `<img src="${albumImg}" alt="" />`
                    : `<div
style="width:40px;height:40px;border-radius:6px;background:rgba(29,185,
84,0.15);display:flex;align-items:center;justify-content:center;flex-sh
rink:0;">
                        <i class="fab fa-spotify"
style="color:#1DB954;"></i>
                    </div>`
                }
                <div class="info">
                    <div class="name">${track.name}</div>
                    <div class="artist">${artistName}</div>
                </div>`;

            item.addEventListener('click', () =>
this.selectTrack(track));
            listEl.appendChild(item);
        });
    }
```

This function handles the **interactive UI rendering** for Spotify search results by dynamically building a list of selectable song tracks. It toggles the visibility of the results container based on data availability and uses `document.createElement` to generate individual track items complete with album art and artist metadata. Each rendered item is then assigned a click event listener, allowing users to select a specific song to associate with their post.

7. Render User profile

```
// ── Render profile header ──────────────────────────────
function renderProfile(user) {
  const displayName = user.display_name || user.username || "Unknown";
  const handle      = `@${user.username ||
displayName.toLowerCase().replace(/\s+/g, "")}`;
  const initials    = getInitials(displayName);

  const avatarEl = document.getElementById("profileAvatar");
  if (avatarEl) {
    if (user.avatar_url) {
      avatarEl.innerHTML = `<img src="${escapeHTML(user.avatar_url)}"
alt="${escapeHTML(displayName)}" />`;
    } else {
      avatarEl.textContent = initials;
    }
  }

  const dnEl  = document.getElementById("profileDisplayName");
  const hdEl  = document.getElementById("profileHandle");
  const bioEl = document.getElementById("profileBio");
  if (dnEl)  dnEl.textContent  = displayName;
  if (hdEl)  hdEl.textContent  = handle;
  if (bioEl) bioEl.textContent = user.bio || "No bio yet. Tell the BUP
community something about yourself!";

  const badgesEl = document.getElementById("profileBadges");
  if (badgesEl) {
    badgesEl.innerHTML = [
      user.course
        ? `<span class="badge badge-course"><i class="fas
fa-graduation-cap"></i> ${escapeHTML(user.course)}</span>`
        : "",
      user.year
        ? `<span class="badge
badge-year">${escapeHTML(user.year)}</span>`
        : "",
      `<span class="badge badge-verified"><i class="fas
fa-check-circle"></i> BU Polangui</span>`,
```

```
    ].join("");
  }
}

// ── Render stats ──────────────────────────────────────────
function renderStats(postsCount, totalLikes) {
  const set = (id, val) => {
    const el = document.getElementById(id);
    if (el) el.textContent = val ?? "—";
  };
  set("statPosts", postsCount);
  set("statLikes", totalLikes);
}

// ── Render posts tab ──────────────────────────────────────
function renderPosts(posts, user) {
  const container = document.getElementById("postsContainer");
  if (!container) return;

  if (!posts.length) {
    container.innerHTML = `
      <div class="feed-empty">
        <i class="fas fa-feather-alt"></i>
        <p>No posts yet. Share something with the BUP community!</p>
      </div>`;
    return;
  }

  const authorName  = user.display_name || user.username || "You";
  const authorInits = getInitials(authorName);

  container.innerHTML = posts.map((p, i) => {
```

This function orchestrates the profile and feed display by mapping user metadata and statistics to the frontend. It dynamically generates profile headers with fallback initials, builds specialized badges for student info, and iterates through a post array to render a personalized activity feed. By managing empty states and verified status icons, it ensures a polished, data-driven representation of the user's presence in the community.

GITHUB COMMITS

REFLECTIONS

James P. Espinosa | Project manager | Documentation Officer

Week 14 is about the display and management of the frontend. In this weeks' activity my task is to prepare a screenshot of the working pages in our project website. Our goal is to make our frontend very user friendly and easy to use. I already added screenshots and as you can see, the frontend web design has a feature of easy accessibility and it is not hard to use. I learned that having a user-friendly website helps the primary users to explore and enjoy the features.

Nadel B. Volante | Database Manager

This week, I monitored all the data that my teammates input during testing of our website to ensure that it is correct and follows the correct format/schema. I also helped in creating the activity/event tracker of our website. I asked for help from both our frontend and backend on how to make the tracker ready for deployment of our website so that it does not cause conflict since I was the one tasked to create the tracker from backend to frontend. I learned that creating even a portion of the website is very difficult and time consuming if not done properly.

Ivan Jacob Milan | Backend Developer

For Week 14, I worked on implementing JavaScript functions to fetch and display data while verifying that the backend GET API endpoints were working correctly. I also connected the frontend to the backend and refactored scripts to make them more modular and organized. On the backend side, I improved the organization system by implementing strict role-based access control, ensuring that only presidents can create organizations and that officer validation is properly secured. I also enhanced the member approval logic to prevent duplicates and unauthorized actions, and prepared the backend structure for future notification tracking for posts, comments, and organization activities.

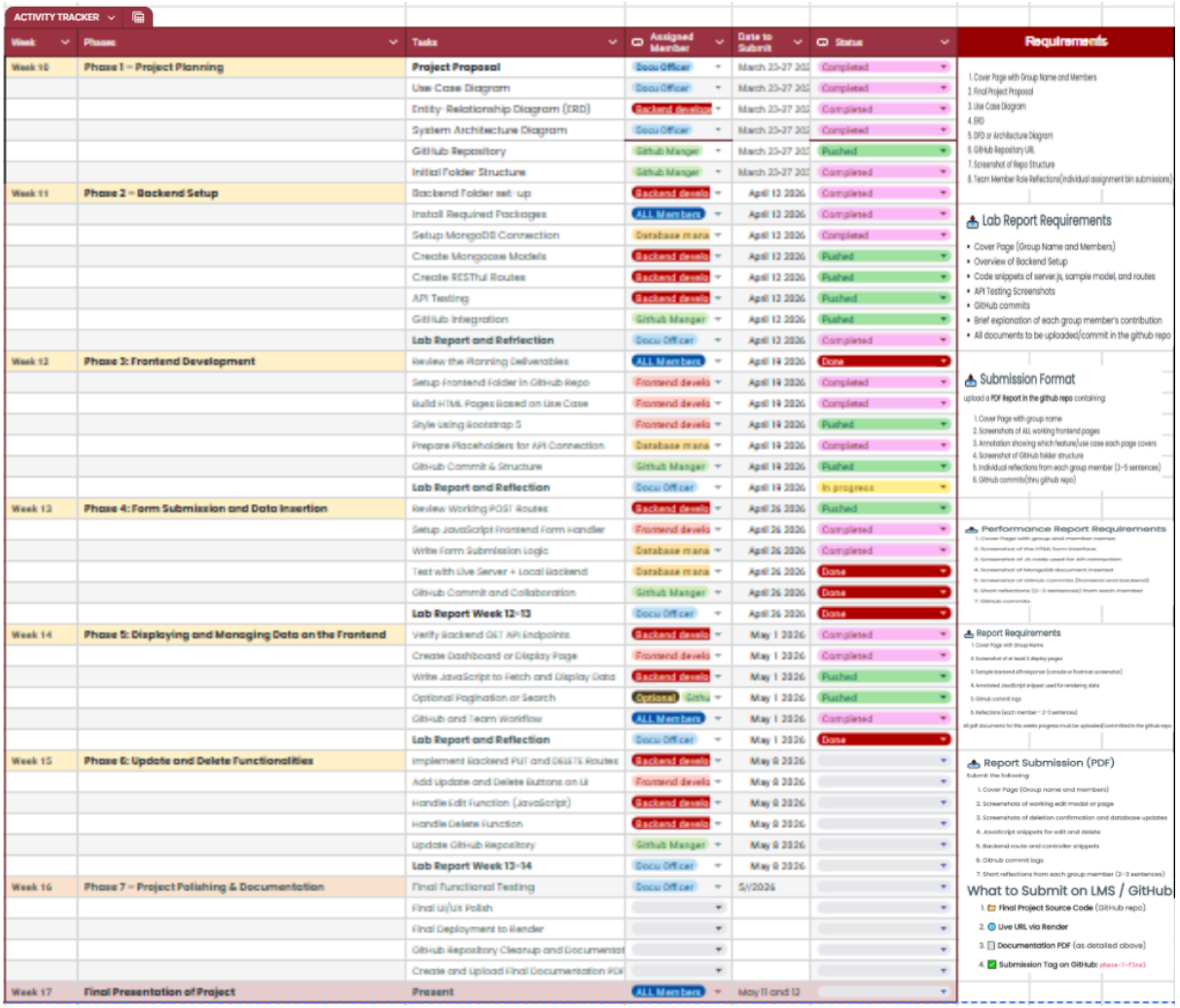
Erick John Bonaobra | Frontend Developer

For week 14, I polished the home, notification and profile tab. In the home and profile tab I added a delete and edit button so users can edit or delete any post they posted. In notification I added a function where if they click a notification, but the post is deleted, at the bottom right a message will pop up saying the post is deleted. I also added a notification badge where

Manuel Angelo A. Loma | GitHub Manager

This week, I contributed to our group project by helping the Frontend Manager with different tasks. One of my main responsibilities was developing the Organization Page, where I focused on making it functional and visually appealing. At the same time, I continued managing our GitHub repository by organizing files, checking updates, and handling branches. This helped our team work more smoothly and avoid conflicts in our code. Even though we faced some issues like merge conflicts and syncing problems, we were able to fix them through teamwork and communication. Overall, this week helped me improve my frontend skills, understand GitHub better, and contribute to our project's progress.

TRACKER UPDATE



ACTIVITY TRACKER

Week	Phase	Tasks	Assigned Member	Date to Submit	Status
Week 10	Phase 1 – Project Planning	Project Proposal	Docu Officer	March 23-27 202	Completed
		Use Case Diagram	Docu Officer	March 23-27 202	Completed
		Entity-Relationship Diagram (ERD)	Backend developer	March 23-27 202	Completed
		System Architecture Diagram	Docu Officer	March 23-27 202	Completed
		GitHub Repository	Github Manger	March 23-27 202	Pushed
		Initial Folder Structure	Github Manger	March 23-27 202	Completed
Week 11	Phase 2 – Backend Setup	Backend Folder set-up	Backend develo	April 12 2026	Completed
		Install Required Packages	ALL Members	April 12 2026	Completed
		Setup MongoDB Connection	Database mana	April 12 2026	Completed
		Create Mongoose Models	Backend develo	April 12 2026	Pushed
		Create RESTful Routes	Backend develo	April 12 2026	Pushed
		API Testing	Backend develo	April 12 2026	Pushed
		GitHub Integration	Github Manger	April 12 2026	Pushed
		Lab Report and Refriection	Docu Officer	April 12 2026	Completed
Week 12	Phase 3: Frontend Development	Review the Planning Deliverables	ALL Members	April 19 2026	Done
		Setup Frontend Folder in GitHub Repo	Frontend develo	April 19 2026	Completed
		Build HTML Pages Based on Use Case	Frontend develo	April 19 2026	Completed
		Style Using Bootstrap 5	Frontend develo	April 19 2026	Pushed
		Prepare Placeholders for API Connection	Database mana	April 19 2026	Completed
		GitHub Commit & Structure	Github Manger	April 19 2026	Pushed
		Lab Report and Reflection	Docu Officer	April 19 2026	In progress
Week 13	Phase 4: Form Submission and Data Insertion	Review Working POST Routes	Backend develo	April 26 2026	Pushed
		Setup JavaScript Frontend Form Handler	Frontend develo	April 26 2026	Completed
		Write Form Submission Logic	Database mana	April 26 2026	Completed
		Test with Live Server + Local Backend	Database mana	April 26 2026	Done
		GitHub Commit and Collaboration	Github Manger	April 26 2026	Done
		Lab Report Week 12-13	Docu Officer	April 26 2026	Done
Week 14	Phase 5: Displaying and Managing Data on the Frontend	Verify Backend GET API Endpoints	Backend develo	May 1 2026	Completed
		Create Dashboard or Display Page	Frontend develo	May 1 2026	Completed
		Write JavaScript to Fetch and Display Data	Backend develo	May 1 2026	Pushed
		Optional Pagination or Search	Optional Githu	May 1 2026	Pushed
		GitHub and Team Workflow	ALL Members	May 1 2026	Completed
		Lab Report and Reflection	Docu Officer	May 1 2026	Done
Week 15	Phase 6: Update and Delete Functionalities	Implement Backend PUT and DELETE Routes	Backend develo	May 8 2026	
		Add Update and Delete Buttons on UI	Frontend develo	May 8 2026	
		Handle Edit Function (JavaScript)	Backend develo	May 8 2026	
		Handle Delete Function	Backend develo	May 8 2026	
		Update GitHub Repository	Github Manger	May 8 2026	
		Lab Report Week 13-14	Docu Officer	May 8 2026	
Week 16	Phase 7 – Project Polishing & Documentation	Final Functional Testing	Docu Officer	5/r2026	
		Final UI/UX Polish			
		Final Deployment to Render			
		GitHub Repository Cleanup and Documentat			
		Create and Upload Final Documentation PDF			
Week 17	Final Presentation of Project	Present	ALL Members	May 11 and 12	

Requirements

1. Cover Page with Group Name and Members
2. Final Project Proposal
3. Use Case Diagram
4. ERD
5. DFD or Architecture Diagram
6. GitHub Repository URL
7. Screenshot of Repo Structure
8. Team Member Role Reflections(individual assignment bin submission)

Lab Report Requirements

- Cover Page (Group Name and Members)
- Overview of Backend Setup
- Code snippets of server.js, sample model, and routes
- API Testing Screenshots
- GitHub commits
- Brief explanation of each group member's contribution
- All documents to be uploaded/commit in the github repo

Submission Format

upload a **PDF Report in the github repo** containing

1. Cover Page with group name
2. Screenshots of ALL working frontend pages
3. Annotation showing which feature/use case each page covers
4. Screenshot of GitHub folder structure
5. Individual reflections from each group member (3–5 sentences)
6. GitHub commits(thru github repo)

Performance Report Requirements

Report Requirements

Report Submission (PDF)

Submit the following:

1. Cover Page (Group name and members)
2. Screenshots of working edit modal or page
3. Screenshots of deletion confirmation and database updates
4. JavaScript snippets for edit and delete
5. Backend route and controller snippets
6. GitHub commit logs
7. Short reflections from each group member (2–3 sentences)

What to Submit on LMS / GitHub

1. **Final Project Source Code** (GitHub repo)
2. **Live URL via Render**
3. **Documentation PDF** (as detailed above)
4. **Submission Tag on GitHub:** phase-7-final

FOOTNOTE:

There are at least 10% use of AI to enhance the explanation made to every code snippet and to better understand the context.